

UF2

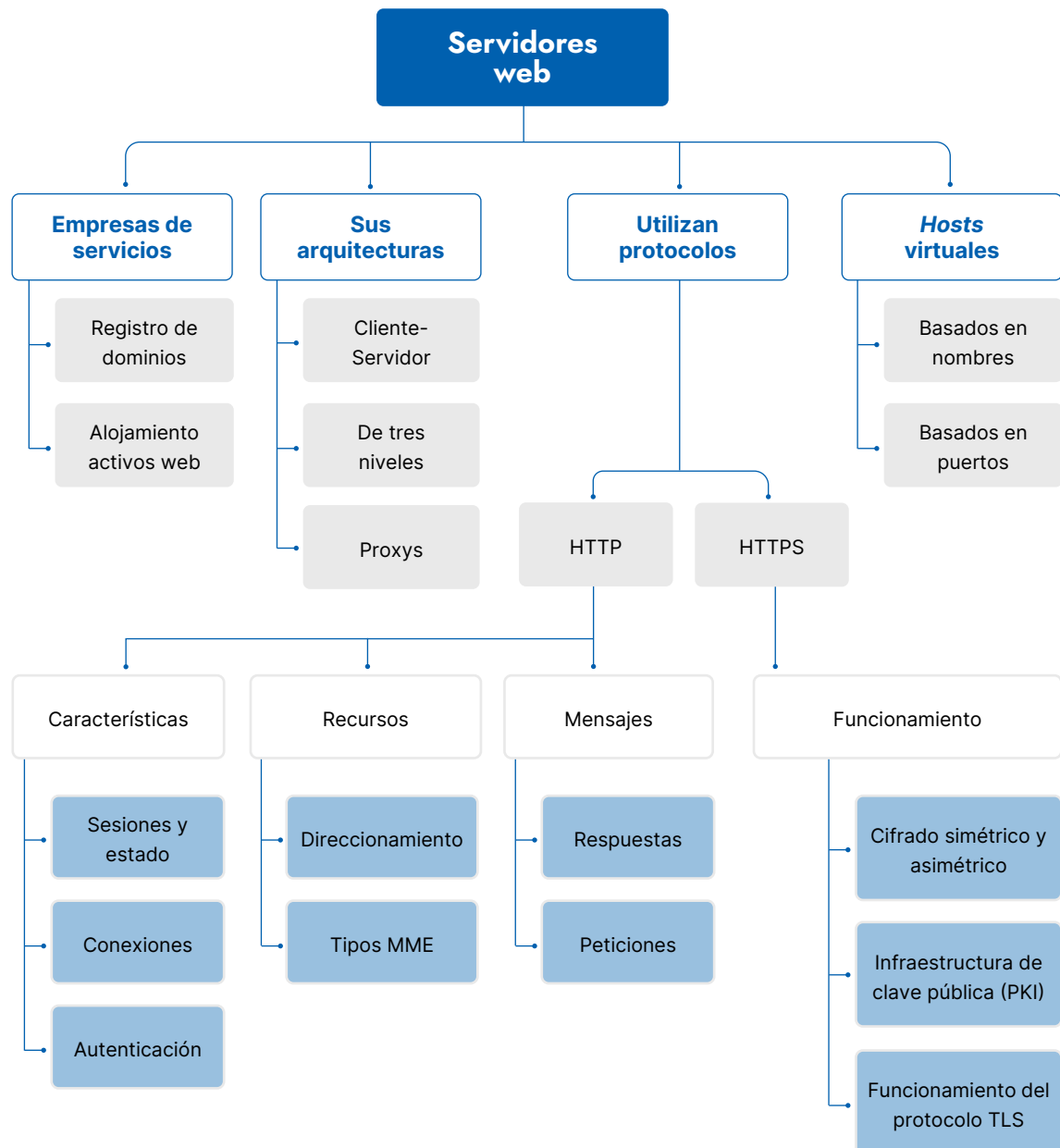
Servidores web

Despliegue de aplicaciones web

ÍNDICE

Mapa conceptual	03
1. Introducción a los servidores web.....	04
1.1. Arquitectura cliente-servidor	04
1.2. Arquitectura de tres niveles	05
1.3. Proxy	06
2. Protocolo HTTP	10
2.1. Historia	10
2.2. Características	11
2.3. Recursos	13
A. Direccionamiento	13
B. Tipos MIME	14
2.4. Mensajes.....	15
A. Peticiones	16
B. Respuestas	18
3. Protocolo HTTPS.....	22
3.1. Funcionamiento	23
A. Cifrado simétrico y asimétrico.....	24
B. Infraestructura de clave pública (PKI)	25
C. Funcionamiento del protocolo TLS	25
4. Hosts virtuales	26
4.1. Hosts virtuales basados en nombres	27
4.2. Hosts virtuales basados en puertos	30
5. Hosting	30

Mapa conceptual



01 Introducción a los servidores web

Dentro de la gran red que es Internet, multitud de dispositivos comparten información gracias a la utilización del protocolo TCP/IP (protocolo de control de transmisión/protocolo de Internet, por sus siglas en inglés). La forma más común de acceder a esta información es mediante la *World Wide Web*, o WWW, que utiliza el protocolo HTTP (*Hypertext Transfer Protocol*).

De entre todos los dispositivos conectados a Internet, hay que distinguir los que tienen la información y los que la solicitan. Los primeros son los llamados **servidores**, ya que están siempre encendidos, en modo escucha, a la espera de recibir una petición de información para buscarla y servirla. Los segundos son los llamados **clientes**, que solicitan la información a los servidores y esperan a recibirla. La ventaja de este modelo es que la información está centralizada en los servidores y los clientes acceden a ella, generalmente, a través de un navegador web, por lo que hacer las copias de seguridad de información y el mantenimiento de los clientes es relativamente sencillo. Esta arquitectura se llama **cliente-servidor**.

En el servidor, los cuatro elementos software más relevantes son: el sistema operativo, el servidor web, el sistema gestor de bases de datos y el lenguaje de programación. La siguiente tabla muestra una comparativa entre los modelos LAMP y WISA; el primero de ellos es libre y el segundo, propietario.

	SISTEMA OPERATIVO	SERVIDOR WEB	SISTEMA GESTOR DE BASES A DATOS	LENGUAJE DE PROGRAMACIÓN
LAMP (LIBRE)	GNU/Linux	Apache	MySQL	PHP
WISA (PROPIETARIO)	Windows	IIS	SQL Server	ASP

Tabla 1.

1.1. Arquitectura cliente-servidor

La arquitectura cliente-servidor se podría representar de la siguiente forma:

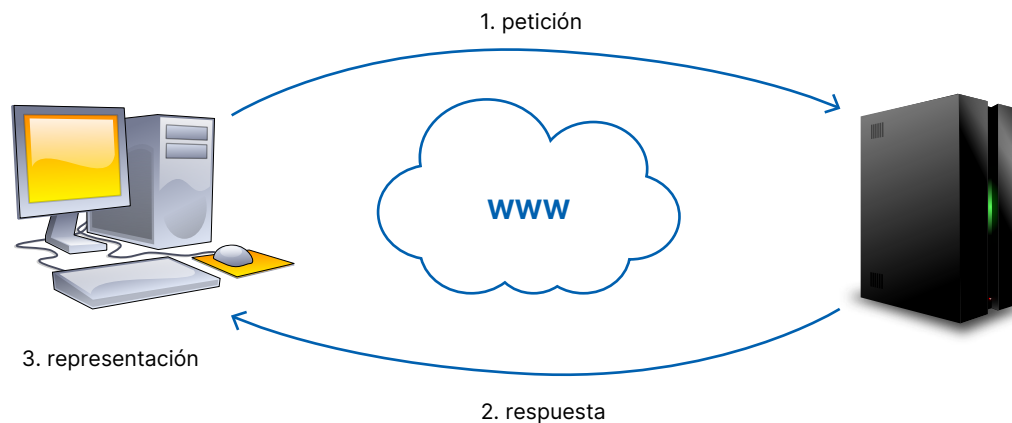


Figura 1. Arquitectura cliente-servidor.

Su funcionamiento se podría describir en los siguientes pasos:

1

El cliente —normalmente, un navegador web— cursa una petición al servidor del recurso que quiere por medio de una URL (*Uniform Resource Locator*).

2

Cuando el servidor ha localizado el recurso, le envía una respuesta al cliente con la información requerida, habitualmente, en formato HTML.

3

En el caso de los navegadores web, los clientes más habituales representan la información HTML en forma de página web para que sea legible por el usuario.

1.2. Arquitectura de tres niveles

En la arquitectura de tres niveles se sigue la misma filosofía que en la arquitectura cliente-servidor, con la diferencia de que el servidor de aplicaciones debe hacer una consulta a una base de datos para completar la información que el cliente le ha solicitado en la petición. El servidor de base de datos puede estar en la misma máquina que el servidor de aplicaciones o en otra distinta. Para el usuario, esto es transparente, ya que es el servidor de aplicaciones quien consulta la base de datos, unifica la información recibida con la suya propia y le manda al cliente el resultado. Este flujo de información se podría representar de la siguiente forma:

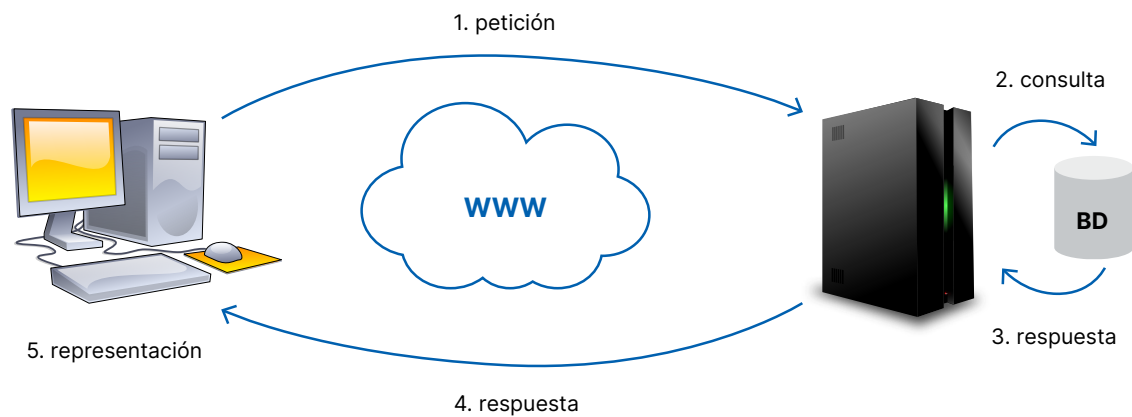


Figura 2. Arquitectua de tres niveles.

Su funcionamiento se podría describir en los siguientes pasos:

- 1** | El cliente, normalmente, un navegador web, efectúa una petición al servidor en la que especifica qué recurso quiere obtener por medio de una URL (*Uniform Resource Locator*).
- 2** | El servidor de aplicaciones le consulta al servidor de bases de datos.
- 3** | El servidor de bases de datos envía la respuesta con la información solicitada al servidor de aplicaciones.
- 4** | El servidor de aplicaciones incorpora la información recibida del servidor de bases de datos a la respuesta que envía al cliente, por lo general, en formato HTML.
- 5** | El cliente representa la información HTML en forma de página web para que sea legible por el usuario.

Un ejemplo de un sitio web que funciona con arquitectura de tres niveles es cualquier CMS, como WordPress, que tiene los archivos PHP en los cuales se integran consultas a la base de datos para obtener la información, que, de manera dinámica, se representa en el navegador del cliente cada vez que se carga una sección del sitio web.

1.3. Proxy

Además del cliente y el servidor, existe otro elemento importante para el buen funcionamiento y la optimización de las comunicaciones HTTP. Se trata del **proxy**, un dispositivo que actúa en la capa de aplicación y hace de intermediario entre el cliente y el servidor. Entre sus funciones están las siguientes:

- » El cacheo de la información, que puede ahorrar tiempo y ancho de banda.
- » El filtrado de contenidos, servidores, etc.
- » El control de acceso mediante la autenticación y los horarios.
- » El balanceo de carga entre distintos servidores.
- » El anonimato de los clientes.

En el siguiente esquema puede verse el funcionamiento básico de un proxy cuando dos clientes le solicitan el mismo recurso susceptible de ser almacenado temporalmente en caché.

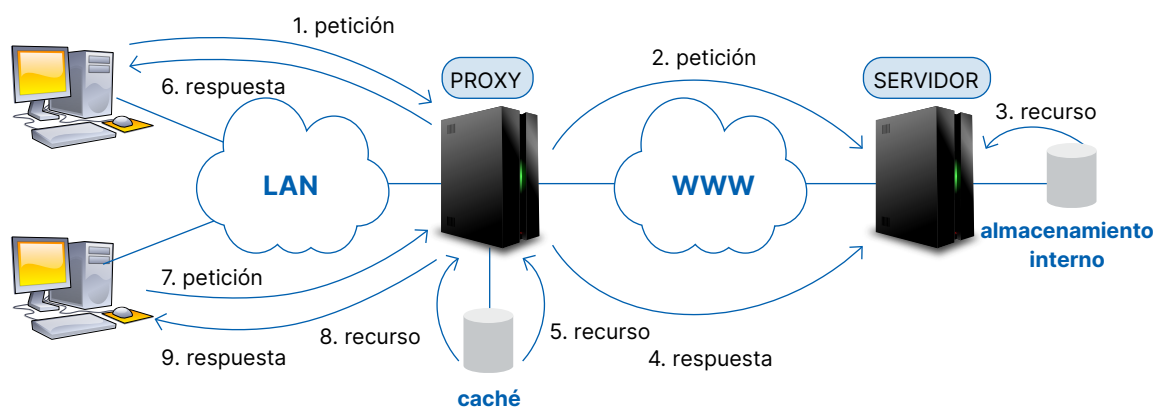


Figura 3. Proxy.

- 1** | El proxy recibe la primera petición por parte de un cliente.
- 2** | La traslada al servidor web.
- 3** | El servidor web localiza el recurso.
- 4** | El servidor envía el recurso al proxy.
- 5** | El proxy almacena el recurso en caché donde lo mantendrá durante un tiempo predeterminado.
- 6** | Además le envía el recurso al cliente que realizó la petición.

7

Si el proxy recibe otra petición del mismo recurso mientras aún se encuentra en la caché, en vez de volver a pedirlo al servidor, lo obtiene de la caché.

8

La caché le proporciona el recurso al proxy.

9

El proxy le sirve el recurso al cliente.

De esta manera, se ahorra tiempo y ancho de banda, sobre todo, cuando se accede a la Web.

1

CASO PRÁCTICO

```
if (strpos($proto_opts, ':') !== false) {  
    list($parsed['hostspec'], $parsed['port']) =  
        explode(':', $proto_opts);  
} else {  
    $parsed['hostspec'] = $proto_opts;  
}  
elseif ($parsed['protocol'] == 'unix') {  
    $parsed['socket'] = $proto_opts;  
}
```

INSTALACIÓN DE LAMP

Como has visto en el documento de teoría, para poder publicar páginas web dinámicas, se necesitan los siguientes elementos: el sistema operativo, el servidor web, el sistema gestor de bases datos y el soporte para uno o varios lenguajes de programación. También viste que todos estos componentes pueden ser libres o propietarios y que LAMP (Linux, Apache, MySQL y PHP) es libre.

En este caso práctico se te pide que instales sobre un sistema operativo Ubuntu Server 22.04 LTS virtualizado el servidor Apache, el sistema gestor de bases de datos MySQL y el soporte de programación para PHP, de manera que la máquina con Ubuntu Server se convierta en un servidor web capaz de alojar sitios web.

Una vez instalado, comprueba el funcionamiento del servidor, sube un archivo index.php a tu sitio web que muestre por pantalla «hola» y comprueba que se muestra correctamente.

SOLUCIÓN

1. Conecta al servidor por SSH:

```
$ ssh admin01@192.168.1.100
```

2. Actualiza la lista de paquetes disponibles en los repositorios y los paquetes del sistema:

```
admin01@srv:~$ sudo apt update -y && sudo apt upgrade -y
```

3. Instala el metapaquete LAMP:

```
admin01@srv:~$ sudo apt install lamp-server^ -y
```

4. Configura el usuario **www-data**, que se crea con la instalación de Apache, para que funcione correctamente:

4.1. Asígnale un *shell* válido:

```
admin01@srv:~$ sudo usermod -s /bin/bash www-data
```

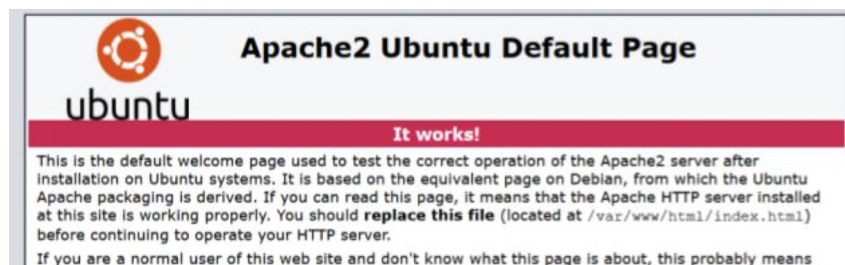
4.2. Asígnale una contraseña:

```
admin01@srv:~$ sudo passwd www-data
```

5. Cambia el usuario y grupo propietarios de la carpeta **/var/www** y todas las carpetas que contiene:

```
admin01@srv:~$ sudo chown -R www-data:www-data /var/www
```

6. Comprueba que el servidor funciona. Al indicar la IP del servidor en la URL del navegador cliente, se debe poder visualizar lo siguiente:



7. Conecta desde otro terminal con el usuario **www-data**:

```
$ ssh www-data@192.168.1.100
```

8. Crea el archivo **index.php** dentro de la carpeta **/var/www/html** con el siguiente contenido:

```
<!DOCTYPE html>
<html>
<head> <meta charset="UTF-8"/> </head>
<body>
<?php
    echo "<h1>hola</h1>";
?>
</body>
</html>
```

9. Borra el archivo **index.html**, ya que tiene preferencia sobre el que se acaba de crear.

02 Protocolo HTTP

La **World Wide Web**, a través del protocolo **HTTP** (*Hypertext Transfer Protocol*) es uno de los pilares básicos de Internet desde la década de los noventa hasta ahora. El protocolo HTTP está pensado para la transferencia de información desde el servidor hacia el cliente en forma de páginas web que, escritas en diferentes lenguajes como **HTML** o XHTML, pueden contener texto, enlaces, formularios, elementos multimedia (como imágenes, animaciones, vídeo, audio...), etc. También pueden incluir scripts y programas que serán ejecutados por el equipo cliente.

Una de las mayores fortalezas de HTTP y de sus protocolos derivados es la capacidad de integrar en la World Wide Web otros servicios como, por ejemplo, el correo electrónico o la mensajería instantánea.

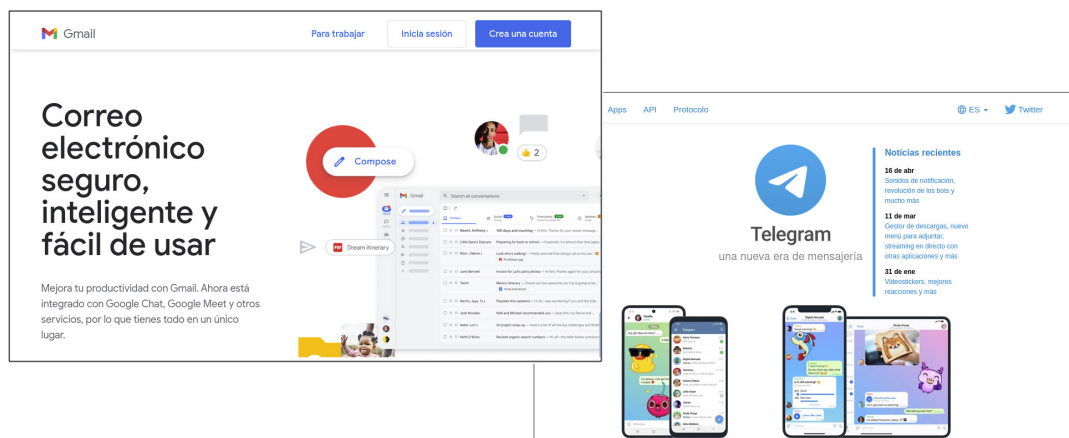


Figura 4. Correo electrónico y HTTP.

HTTP es un protocolo de capa de **aplicación** de la pila de protocolos TCP/IP y que, normalmente, se apoya en el protocolo TCP (protocolo de control de transmisión, por sus siglas en inglés) de la capa de transporte. El puerto de escucha por defecto que usan los servidores web es el 80/TCP.

2.1. Historia

Estos son los hitos más importantes de la evolución del protocolo HTTP desde su creación en 1989 por **Timothy "Tim" John Berners-Lee**:

- » **1989.** Tim Berners-Lee escribió la **primera aproximación a la World Wide Web** y estableció sus principales componentes: **URL**, un sistema que le permite al cliente especificar los recursos a los que quiere acceder; **HTTP**, un protocolo de transferencia que permite transferir los recursos del servidor al cliente, y, por último, **HTML**, el lenguaje de marcas en el que están escritos los recursos, es decir, las páginas web.

- » **1990. Primera comunicación HTTP** entre un cliente y un servidor a través de Internet.
- » **1991.** Se escribe **HTTP/0.9** en dominio público. Es la primera definición de HTTP.
- » **1992.** Se publica HTTP Básico. Es una actualización del protocolo que se definió como borrador interno del **IETF** (*Internet Engineering Task Force*).
- » **1994.** Fundación del **W3C** (*World Wide Web Consortium*), cuyo objetivo es la creación de estándares para el desarrollo de la World Wide Web.
- » **1996.** Se publica la RFC 1945 que define **HTTP/1.0**.
- » **1997.** Se publica **HTTP/1.1** como la RFC 2068.
- » **1999.** La definición de HTTP/1.1 en la RFC 2616 deja obsoleta la RFC 2068.
- » **2014.** Con la publicación de seis documentos RFC, de la RFC 7230 a la RFC 7235, se hace una revisión profunda del protocolo HTTP/1.1 que deja obsoleta la definición recogida en la RFC 2616.
- » **2015.** Se define **HTTP/2** en la RFC 7540. Esta es la versión actual.
- » **2020.** Se publica el borrador de **HTTP/3**.

En esta línea de tiempo se puede ver un resumen en el que únicamente aparecen las fechas más relevantes de las distintas versiones del protocolo HTTP:

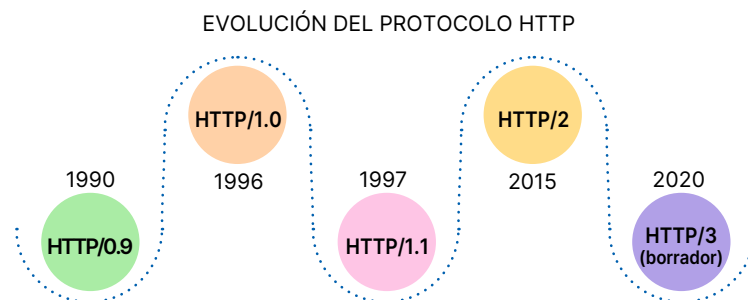


Figura 5. Evolución del protocolo HTTP.

2.2. Características

En este apartado se describen algunas de las características del protocolo HTTP.

Sesiones y estado

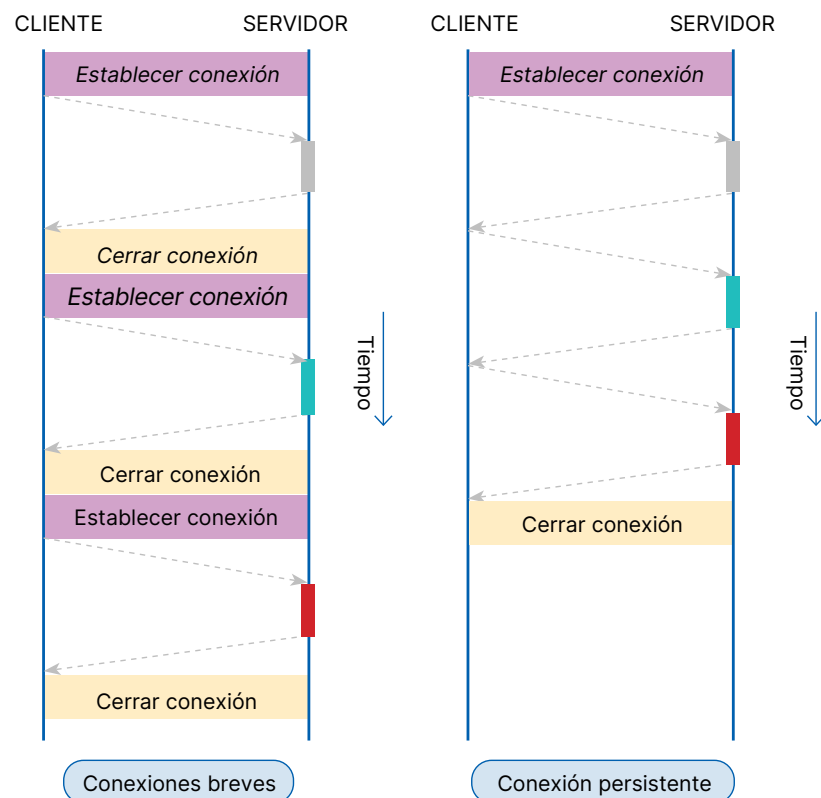
La actividad que realiza un usuario sobre una página de un servidor web es una sesión. Como HTTP es un protocolo sin estado, no guarda información entre las distintas peticiones, así que se deben usar una serie de técnicas para lograr recoger la interacción que tiene el usuario con el sitio web durante toda la sesión. Una de estas técnicas es el uso de las cookies con las que se pueden crear aplicaciones web que permitan llevar un seguimiento de las acciones de cada uno de los usuarios.

A

Conexiones

Las versiones antiguas de HTTP establecen una conexión TCP de capa de transporte por cada petición y respuesta entre el cliente y el servidor, lo cual incrementa el tráfico entre los equipos, así como el tiempo invertido. Se trabaja con lo que se llaman conexiones breves.

Para mejorar el rendimiento de las comunicaciones HTTP e intentar reducir los efectos de este problema, a partir de HTTP/1.1 se permiten las conexiones persistentes que permiten varias comunicaciones HTTP por cada conexión TCP. De esta manera, si una página web realiza varias peticiones de recursos a un mismo servidor, solo se establecerá una conexión TCP.



Por otra parte, a partir de la versión HTTP/2, se pueden multiplexar diferentes mensajes HTTP sobre la misma conexión TCP, lo que aumenta sensiblemente el rendimiento de las comunicaciones.

B

Autenticación

HTTP permite que el servidor establezca un sistema de autenticación de los clientes y control de acceso sobre los recursos del servidor, de manera que el cliente facilitará unas credenciales que serán comprobadas por el servidor o un proxy para determinar si le da acceso o no a ciertos recursos. Existen varios esquemas de autenticación, entre ellos:

- » Autenticación *Basic*: el cliente facilita el usuario y la contraseña al servidor. Esta última puede enviarse codificada en base64, encriptada o aplicando una función *hash*. Cuando la contraseña se transmite con un formato inseguro, debe usarse una conexión HTTPS (TLS) para poder considerarlo un método seguro.
- » Autenticación *Digest*: se transmite el resultado de aplicar la función MD5 tanto al nombre de usuario como a la contraseña.
- » Protocolo *OAuth*: el propietario de un recurso en un servidor permite el acceso de un tercero a dicho recurso sin tener que facilitarle sus credenciales. El permiso que otorga el propietario del recurso al cliente que quiere acceder a su recurso se basa en un *token* y en una contraseña. Este sistema permite, por ejemplo, que aplicaciones de terceros como, por ejemplo, Kami, accedan a los archivos PDF almacenados en Google Drive para realizar anotaciones sobre ellos.

En la actualidad no es común encontrar servidores web que gestionen la autenticación directamente. La tendencia es delegar esta función a las aplicaciones web instaladas.

C

Esquema 1.

2.3. Recursos

A. Direccionamiento

Uno de los pilares básicos de la World Wide Web es el direccionamiento de los recursos, es decir, cómo se identifican. La Web usa un sistema común de identificación global basado en los **URI** (*Uniform Resource Identifier*), que son cadenas de caracteres con una determinada sintaxis que identifican los recursos disponibles. Existen dos tipos de URI: el **URL** (*Uniform Resource Locator*), que identifica un recurso por medio de su localización, y el **URN** (*Uniform Resource Name*), que lo identifica por medio de su nombre, pero sin determinar dónde se encuentra. De estos dos tipos de URI, el URL es el tipo más usado en la World Wide Web, de hecho, se conoce más comúnmente como dirección web. Así, al utilizar este direccionamiento, los clientes especifican sin lugar a dudas los recursos que quieren obtener. El resto de los elementos de la arquitectura web deben reconocer el sistema de direccionamiento para poder proporcionar dichos recursos sin equivocaciones. La estructura del localizador de recursos uniforme (URL) recogida en el documento RFC 3986 es:

esquema": "[//"**autoridad**]"camino["?"**consulta**]"

- » El **esquema** indica el protocolo usado para localizar el recurso, por ejemplo, http, ftp o mailto. Va seguido del carácter ":".
- » La **autoridad** hace referencia al servidor y puede omitirse. La *estructura* de la autoridad es `[info_de_usuario@"host[:puerto]`, donde se puede incluir información del usuario usado, la dirección IP o el nombre del servidor y el puerto de escucha. Va precedido de los caracteres "//".
- » El **camino** indica la localización exacta del recurso dentro del equipo y su nombre.
- » La **consulta**, también llamada parámetros, es un conjunto de pares `clave="valor` separados por el carácter "&", que son capaces de mandar información desde el cliente al servidor. El carácter "?" marca el inicio de la consulta.

Nota: Los elementos entre corchetes son opcionales.

Estos son algunos ejemplos de URL:

`https://datatracker.ietf.org/doc/html/rfc3986#section-3`

`https://aules.edu.gva.es:443/fp/course/view.php?id=14908`

B. Tipos MIME

Los tipos **MIME** (*Multipurpose Internet Mail Extensions*) permiten gestionar correctamente la información de los correos electrónicos de manera que se pueda, por ejemplo, incluir texto en codificaciones diferentes a US-ASCII y otros formatos no textuales, como imágenes, audio, vídeo, etc. Estas extensiones han tenido mucho éxito, por lo que no es de extrañar que, en el contexto del protocolo HTTP, sea muy común que los distintos elementos que participan en la comunicación usen los tipos MIME en vez de las extensiones de los archivos para conocer su tipo y poder tratarlos correctamente.

Los tipos MIME tienen la siguiente estructura:

`tipo/[árbol.]subtipo[+sufijo] [;parámetros]`

Aunque lo normal es encontrarlos con la forma abreviada `tipo/subtipo`. Por ejemplo, `image/png` indicaría que se trata de una imagen PNG.

IANA (*Internet Assigned Numbers Authority*), la organización que se encarga de la coordinación mundial de los DNS raíces, del direccionamiento IP y de los tipos MIME, define los tipos principales y todos los posibles subtipos. En la siguiente tabla se recogen algunos de ellos.

TIPO MIME	EJEMPLOS TIPO/SUBTIPO	COMENTARIOS
Application	application/octet-stream application/pdf	Archivos binarios en general, ya sean ejecutables o no, archivos pdf, etc.
Audio	audio/ogg audio/mpeg	Archivos de audio.
Font	font/ttf	Formatos de fuentes.
Image	image/gif image/jpeg	Imágenes
Message	message/http message/rfc822	Mensajes.
Model	model/x3d+xml	Modelos gráficos en 3D.
Multipart	multipart/form-data multipart/signed	Se usa para documentos compuestos.
Text	text/html text/css	Texto sin código binario.
Video	video/ogg video/mpeg	Vídeo.

Tabla 2. Tipos de MIME.

2.4. Mensajes

Como ya se ha visto, la transferencia de información del protocolo HTTP se basa en el intercambio de mensajes entre el cliente y el servidor web. Existen dos tipos de mensajes: las **peticiones** y las **respuestas**. Las peticiones las realiza el cliente para indicarle al servidor qué recurso quiere obtener, mientras que las respuestas son los mensajes que usa el servidor para atender las peticiones del cliente.

Aunque el formato de los mensajes se ha mantenido relativamente estable en las distintas versiones del protocolo, HTTP/2 introduce algunas mejoras relacionadas directamente con la gestión de los mensajes y la reducción de las comunicaciones, así como con su optimización:

- » **Transmisión** de los mensajes **en código binario**, por lo que el tratamiento en la red es más sencillo, tiene menos errores y se incrementa el rendimiento de las comunicaciones entre el cliente y el servidor.
- » **Compresión** de las cabeceras de los mensajes, por lo que se reduce la cantidad de información enviada.
- » **Eliminación de la información redundante** de las cabeceras de varios de los mensajes de una misma comunicación.
- » **Multiplexación** de los mensajes entre un mismo cliente y el servidor web.
- » La técnica **server push**, que le permite al servidor hacer una previsión de qué recursos necesitará el cliente. Así podrá enviar varias respuestas tras recibir una única petición.
- » **Fragmentación** de los mensajes en varias partes que serán enviadas por separado entre los equipos intervinientes en la comunicación.
- » **Priorización** de los fragmentos en los que se dividen los mensajes.

El formato de los mensajes, ya sean peticiones o respuestas, es muy parecido. De hecho, solo difieren en la primera línea. Aunque en la versión HTTP/2 los mensajes se transmiten en binario, al igual que HTTP/1.1, están compuestos de caracteres ASCII divididos en diferentes líneas. Ambos tipos de mensajes pueden tener dos partes: la **cabecera** (*header*) y el **cuerpo** (*payload*).

A. Peticiones

La **petición** es el mensaje que le envía un cliente al servidor para indicarle qué recurso quiere obtener. La estructura de su cabecera es la siguiente:

- » Una **línea de petición**, que está compuesta por el **método** o acción que el cliente quiere que se realice, la URL del recurso y la versión del protocolo HTTP que hay que usar.
- » De forma opcional, un conjunto de campos de cabecera que pueden contener opciones, información sobre el cliente que efectúa la petición y otros metadatos. Cada uno de estos campos se escribe en una línea.
- » Y el **fin de la cabecera**, que, simplemente, es una línea en blanco.

A continuación, se presenta un **ejemplo** de la cabecera de una petición:


```
GET /tabla/tabla.html HTTP/1.1
Accept: text/html,application/xhtml+xml,application/xml,image/avif
Accept-Encoding: gzip, deflate
Accept-Language: es-ES,es;q=0.8,en-US;q=0.5,en;q=0.3
Host: 192.168.202.132
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:96.0)
Gecko/20100101 Firefox/96.0
Connection: keep-alive
```

Como se puede ver en la primera línea, aparece el método *GET*, que le indica al servidor que tiene que enviar el recurso *tabla.html* al cliente a través del protocolo *HTTP/1.1*. Después aparecen los campos de cabecera y, por último, una línea en blanco que indica el fin de la cabecera.

En este ejemplo, los campos de cabecera indican:

- » Los tipos MIME soportados,
- » Los tipos de compresión admitidos.
- » Los idiomas soportados por el cliente.
- » La dirección IP del servidor.
- » Información acerca del cliente web usado.
- » El tipo de conexión, que, en este caso, sería persistente.

Por último, se deja una línea en blanco.

Se puede obtener una lista mucho más amplia de los campos de cabecera en la página web oficial de IANA, sección *Protocol Assignments*, epígrafe *Message Headers*.

Como ya se ha comentado, el **método** es la acción que el cliente quiere realizar sobre el recurso especificado en la URL. Estos son algunos de los métodos que existen, aunque solo los dos primeros son obligatorios en cualquier servidor web de propósito general:

MÉTODO	DESCRIPCIÓN	DEFINICIÓN
GET	Transferir el recurso indicado del servidor al cliente. Es el método más común.	RFC 7231
HEAD	El servidor solo envía al cliente la cabecera. Se usa para obtener los metadatos del recurso sin tener que transferirlo y para realizar pruebas de funcionamiento.	RFC 7231
POST	Envía información al recurso indicado para que la procese. Uno de sus usos más comunes es pasar la información recogida en un formulario HTML para que un script del servidor la procese.	RFC 7231

PUT	Envía información al servidor para que cree o actualice el contenido de un recurso.	RFC 7231
DELETE	Borra un recurso del servidor.	RFC 7231
CONNECT	Se establece un túnel con el servidor indicado en la petición. Normalmente, se usa para crear conexiones de extremo a extremo con uno o más proxies.	RFC 7231
OPTIONS	Solicita las opciones disponibles en el servidor para un determinado recurso.	RFC 7231
TRACE	El cliente usa este método para probar el camino desde el cliente hasta el servidor, de manera que espera la respuesta del servidor.	RFC 7231
PATCH	Se usa para modificar parcialmente un recurso.	RFC 5789

Tabla 3. Tipos de peticiones.

Los documentos RFC 2518 y RFC 3253 recogen métodos adicionales.

B. Respuestas

La **respuesta** es cada uno de los mensajes enviados desde el servidor contestando a un cliente que le ha cursado una petición, por lo que se deduce que la petición de un cliente puede tener más de una respuesta del servidor. La estructura de la cabecera de una respuesta es como sigue:

- » Una **línea de estado**, compuesta por la versión del protocolo HTTP; un código que, dependiendo del resultado del intento de obtención del recurso, indicará que se ha conseguido o que ha habido un error, y una descripción del código.
- » Opcionalmente, un conjunto de **campos de cabecera**, cada uno en una línea diferente, que pueden contener información sobre el servidor, datos sobre el recurso y otros metadatos.
- » Y el **fin de la cabecera**, que, simplemente, es una línea en blanco.

En este otro **ejemplo** se recoge la cabecera de una respuesta:

```
HTTP/1.1 200 OK
Date: Wed, 08 Jun 2022 06:40:45 GMT
Server: Apache/2.4.52 (Ubuntu)
Last-Modified: Mon, 23 May 2022 17:52:43 GMT
Accept-Ranges: bytes
Content-Encoding: gzip
Content-Length: 225
Connection: Keep-Alive
Keep-Alive: timeout=5, max=100
Content-Type: text/html
```

En la línea de estado se puede ver el protocolo usado y el estado de la petición. En este caso, se indica el código *200*, que significa que todo ha ido correctamente. Después, al igual que en la cabecera de la petición, aparecen los campos de cabecera y una línea en blanco que marca el fin de la cabecera.

Los campos de cabecera de este ejemplo indican, en orden de aparición:

- » La fecha en la que se atendió la petición.
- » Información sobre el servidor web.
- » Cuándo se modificó por última vez el recurso.
- » Que el cliente debe usar los bytes como unidad de medida en caso de realizar peticiones parciales del recurso.
- » Que el cuerpo del mensaje se ha codificado en formato *gzip*.
- » El tamaño del cuerpo, medido en bytes.
- » Que la conexión será de tipo permanente.
- » Los parámetros de configuración de las conexiones permanentes.
- » El tipo MIME del recurso enviado en el cuerpo.

Tal como se ha dicho antes, las líneas de estado de las cabeceras de los mensajes de respuesta recogen **el código y la descripción del estado** de la petición. El código es un número de tres dígitos donde el primero indica la clase de respuesta, mientras que los dos últimos indican la respuesta concreta. A continuación, se muestra una tabla con los tipos de respuesta y algunos ejemplos.

1XX	RESPUESTAS INFORMATIVAS
100	Continue: indica que la petición se está tramitando correctamente, pero aún no ha acabado.
2xx	RESPUESTAS SATISFACTORIAS
200	Ok: la petición se ha tramitado correctamente.
201	Created: se ha creado uno o más recursos a raíz de la petición.
202	Accepted: la petición se ha aceptado, pero aún no se ha podido procesar, por lo que tampoco se puede saber si tendrá éxito o no.
3XX	REDIRECCIONAMIENTOS
300	Multiple choices: se indica que la petición tiene varias posibles respuestas y será el usuario o el cliente quien debe seleccionar una de ellas.
301	Moved permanently: el recurso pedido se ha cambiado de localización, por lo que se envía una nueva URL para que el cliente pueda pedirlo de nuevo.
302	Found: el recurso pedido ha cambiado temporalmente de localización.
4XX	ERROR POR PARTE DEL CLIENTE
400	Bad Request: el servidor no puede procesar la petición por un error por parte del cliente.
403	Forbidden: el cliente ha solicitado un recurso sobre el que no tiene permisos para realizar la operación indicada.
404	Not Found: no existe el recurso solicitado o el servidor no quiere revelar su existencia.
5XX	ERROR POR PARTE DEL SERVIDOR
500	Internal Server Error: indica que el servidor ha encontrado un error interno que le impide llevar a cabo la operación solicitada.
501	Not Implemented: el servidor no tiene implementada la operación solicitada.
502	Bad Gateway: indica que un proxy ha recibido una respuesta inválida de un servidor.
503	Service Unavailable: el servidor no puede satisfacer la petición en ese momento por una sobrecarga de trabajo o una parada programada de mantenimiento.
504	Gateway Timeout: el proxy no ha obtenido respuesta del servidor.

Tabla 4. Tipos de respuestas.

2 CASO PRÁCTICO



https://

CABECERAS HTTP

Como ya sabes, dentro de la cabecera de las peticiones y respuestas HTTP puede haber diferentes campos de cabecera. En esta actividad vas a ver para qué sirven o qué significan algunos de los campos de cabecera que se generan y, así, averiguar qué información sobre tu sistema, o sobre ti, se envía al servidor cuando haces una petición.

Para realizar esta tarea, usarás las herramientas de inspección de tu navegador web, herramientas on-line o programas específicos.

SOLUCIÓN

Para consultar las cabeceras se pueden utilizar diferentes tipos de herramientas. Usa al menos dos de estas opciones:

- » **Inspector de Firefox 100.0.2:** abre el menú contextual de la página usando el botón derecho del ratón sobre ella, accede a la opción *Inspección > Pestaña Red*, asegúrate de que la casilla de verificación *Desactivar caché* está marcada, recarga la página y haz clic sobre las peticiones para ver los detalles en el panel lateral derecho. Si se activa el control *Sin procesar*, verás la cabecera original.
- » **<https://headers.cloxy.net/>:** accede a esta página e introduce la URL completa de la página para inspeccionar en el cuadro de texto y pulsa el botón *Get Headers*. Se muestran las cabeceras de respuesta. Para ver la que corresponde a la petición, accede al enlace *View your request headers*.
- » **Wireshark:** ejecuta este programa con permisos de superusuario en el mismo ordenador en el que usarás el cliente web, haz clic sobre *Capture*, selecciona la interfaz de red adecuada y pulsa el botón *Start*. Abre el cliente web y accede a una página. Vuelve a Wireshark, pulsa el botón *Stop capturing packets*, localiza el primer paquete HTTP en la lista de paquetes y selecciónalo; puedes ayudarte del filtro de paquetes o de la columna *Protocol*. Accede al panel central donde se muestran los detalles del paquete seleccionado y despliega el elemento *Hypertext Transfer Protocol* para ver la cabecera de la petición.

Una vez obtenidas las cabeceras de las peticiones, se pueden localizar algunos campos que ofrecen cierta información sobre el sistema del cliente y sus preferencias personales. Por ejemplo:

- » *User-Agent*: muestra información sobre tu cliente web.
- » *Sec-CH-UA-Platform*: plataforma o sistema operativo desde el que se ejecuta el cliente.
- » *Accept-Language*: idiomas aceptados y cuales se prefieren.

Ten en cuenta que, dependiendo del cliente usado, pueden aparecer unas cabeceras u otras.

03 Protocolo HTTPS

HTTPS corresponde a las siglas en inglés *Hypertext Transfer Protocol Secure*, aunque, realmente, no se trata de un protocolo, sino del uso de HTTP sobre el protocolo **TLS** (*Transport Layer Security*) o, en sus orígenes, sobre **SSL** (*Secure Sockets Layer*), el antecesor de TLS. De esta manera se introduce un nuevo protocolo entre HTTP y TCP que implementa un canal cifrado entre cliente y servidor, de manera que incrementa la **seguridad** en las comunicaciones en forma de:

- » **Autenticación**: el servidor siempre se autentica frente al cliente, mientras que en sentido contrario es opcional.
- » **Confidencialidad**: la información transferida solo la conocen los intervinientes en la comunicación.
- » **Integridad**: se detectan los cambios accidentales o deliberados en la información.

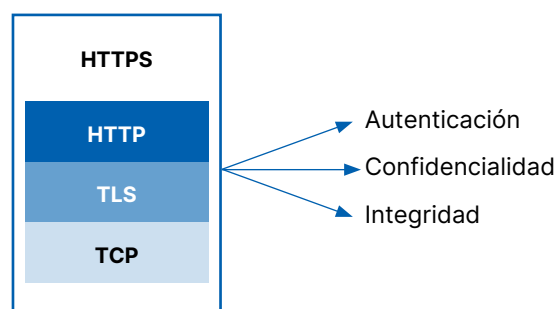
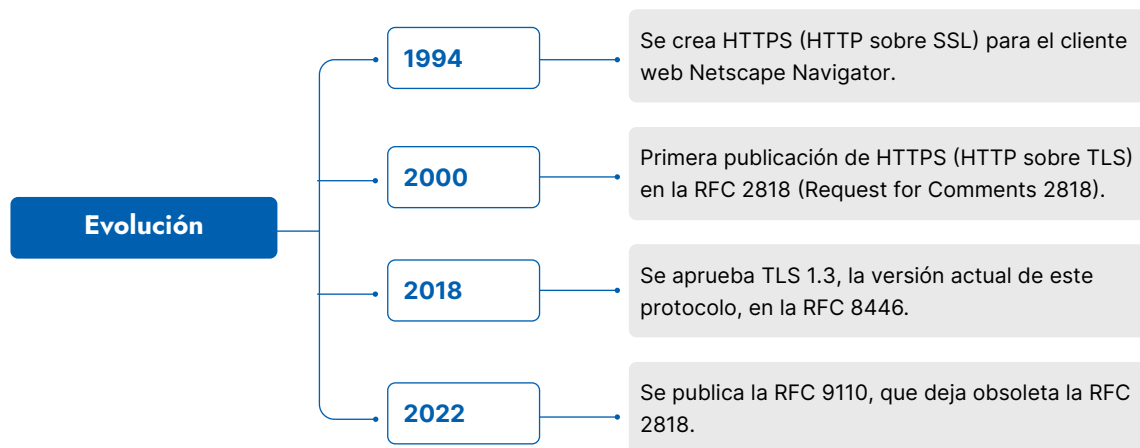


Figura 6. tipos de protocolos HTTPS.

Dejando de lado la seguridad aplicada, las diferencias entre HTTP y HTTPS son pocas. Estas son las más significativas:

- » Tienen prácticamente la misma estructura de localizador. Solo cambia el esquema que toma el valor `http` o `https`, según sea el protocolo usado.
- » El puerto de escucha por defecto del servidor HTTPS es el 443, mientras que en HTTP es el 80. En ambos protocolos se puede usar un puerto diferente si se configura de forma correcta.



Según W³Techs, actualmente, la mayoría de los sitios web usan, al menos en parte, el protocolo HTTPS. Este es un extracto de los datos publicados en junio de 2020 en el que se observa el porcentaje de sitios web que usan HTTPS frente a los que usan versiones no seguras de HTTP. Hay que tener en cuenta que un sitio web puede usar más de un protocolo.

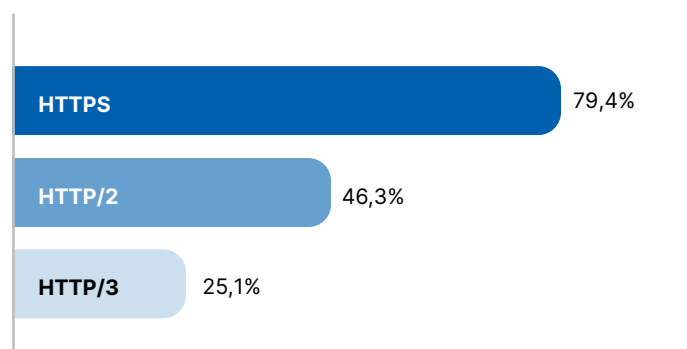


Figura 7. Evolución de HTTPS.

3.1. Funcionamiento

Como ya se ha dicho, HTTPS es el resultado de aplicar HTTP sobre una conexión TLS que protege la conexión, por lo que lo estudiado para el protocolo HTTP es aplicable directamente a HTTPS. A este funcionamiento hay que añadirle lo concerniente a la capa de seguridad extra aportada por TLS, por lo que este apartado se centra en el funcionamiento de este último protocolo, basado en el cifrado híbrido, que combina el cifrado simétrico con el asimétrico, y en los certificados.

A. Cifrado simétrico y asimétrico

El **cifrado simétrico** usa la misma clave, o contraseña, para cifrar y descifrar la información, por lo que, si se quiere transmitir, tanto el emisor como el receptor deben conocer cuál es la contraseña. Las *ventajas* de este tipo de cifrado son la rapidez y la facilidad de implementación, mientras que, entre sus *desventajas*, por un lado, está la interceptación de las claves cuando se comparten entre emisor y receptor y, por otro lado, el uso de claves poco robustas, ya que pueden averiguarse fácilmente por ataques de fuerza bruta, de diccionario o de ingeniería social. Algunos de los algoritmos de cifrado simétrico son AES, 3DES, Blowfish...

El **cifrado asimétrico** se caracteriza por el uso de pares de claves, la pública y la privada, para cifrar y descifrar la información. En cuanto a las claves, hay que decir lo siguiente:

- » Las claves se generan a partir de números primos muy grandes y no se pueden repetir.
- » Mientras que la clave privada se debe mantener en secreto y nunca se debe compartir, con la clave pública es justamente al revés, cuanto más personas la conozcan, mejor.
- » Toda información cifrada con una clave solo puede ser descifrada con la correspondiente clave asociada, ni siquiera con la misma clave usada para cifrar. Es decir, si un mensaje está cifrado con una clave pública, solo su correspondiente clave privada podrá descifrar el mensaje, y viceversa.
- » La privacidad de las comunicaciones se consigue cuando el emisor cifra el mensaje con la clave pública del receptor, por lo que solo se podrá descifrar con la clave privada del receptor. Además de la privacidad, se garantiza la integridad de la información, es decir, que no ha sufrido ningún cambio.
- » Para garantizar la identidad del emisor de un mensaje, el emisor la cifra con su propia clave privada, por lo que el receptor, si es capaz de descifrarlo con la clave pública del emisor, puede estar seguro de que el mensaje se ha enviado por el emisor y que no ha sufrido alteraciones por el camino.
- » Cuando se cifra con la clave privada de uno mismo, no se mantiene la confidencialidad, pero se **firma** digitalmente una información, lo que garantiza la identidad (autenticación) y el no repudio.

Entre las *ventajas* del cifrado simétrico, encontramos el alto nivel de seguridad y la poca cantidad de claves que se deben gestionar, solo dos por interviniente. Por otro lado, su principal *desventaja* es el alto coste computacional, lo que, a su vez, aumenta el tiempo de cifrado y descifrado. Algunos ejemplos de algoritmos de cifrado asimétrico son Diffie-Hellman, ElGamal, de curvas elípticas, DSA, RSA...

B. Infraestructura de clave pública (PKI)

El uso del cifrado asimétrico para la comunicación segura en entornos reducidos no plantea muchas dudas a la hora de confiar en la propiedad real de las claves públicas que se obtienen. El problema llega cuando se usa en entornos de cierta envergadura, sobre todo, cuando se trata de sistemas globales como la World Wide Web. En estos casos es necesario establecer algún tipo de mecanismo que permita establecer cierta confianza cuando se obtienen las claves públicas de personas, organizaciones, servidores, etc. Un ejemplo muy significativo de estos mecanismos es la **infraestructura de clave pública, o PKI**, que se basa en hardware, software, procedimientos y relaciones de confianza entre los distintos actores que intervienen. A continuación, se presentan de manera simplificada algunos de estos actores y sus funciones:

- » **Certificado:** clave pública de una entidad que se ha firmado con la clave privada de una autoridad certificadora. Los certificados se pueden usar tanto para firmar digitalmente como para cifrar.
- » **Autoridad de certificación (AC):** entidad en la que los usuarios depositan su confianza, de manera que, cuando la AC genera un certificado, todos los que confían en la AC asumen que el certificado pertenece a ese usuario.
- » **Usuario:** persona, organización, servidor, etc., que puede ser el propietario de los certificados, el que quiere verificar la validez de un certificado o ambas cosas.

Uno de los usos más extendidos de los certificados es la **validación de los servidores web frente a los clientes**, de manera que, si el servidor tiene un certificado firmado por una de las AC en la que el cliente confía, se asumirá que se ha conectado al servidor original y que no ha sido suplantado por otro fraudulento. La mayoría de los clientes web tienen una lista predefinida de autoridades de certificación en las que confían que puede modificar el usuario. Además de la lista de AC, también cuentan con otras listas que contienen, por ejemplo, certificados de servidores, los suyos propios y los de otros usuarios.

C. Funcionamiento del protocolo TLS

Como ya se ha dicho, **TLS utiliza el cifrado híbrido** para asegurar las comunicaciones entre un cliente y un servidor configurado con el protocolo HTTPS, lo que significa que utilizará tanto el cifrado simétrico como el asimétrico. De esta manera paliará las carencias de cada tipo de cifrado con las virtudes del otro. Para cifrar la información, se usará el cifrado simétrico porque es más rápido, mientras que se usará el cifrado asimétrico para evitar que un posible atacante obtenga las claves simétricas.

TLS 1.3 tiene dos componentes:

»El **protocolo Handshake**: es el primer paso y se encarga de establecer los parámetros que regirán la comunicación. Estas son sus fases:

- Fase 1, o **intercambio de claves**: establecimiento de las claves compartidas entre el cliente y el servidor. Estas claves se usarán para cifrar el recurso pedido.
- Fase 2, o **parámetros de servidor**: decidir los parámetros que usará el servidor en la comunicación, como, por ejemplo, los algoritmos criptográficos y las claves simétricas que se usarán.
- Fase 3, o **autenticación**: autenticación del servidor frente al cliente a través del certificado digital y, si así se establece, del cliente frente al servidor.

»El **protocolo de registro**, que aplica los parámetros indicados por el protocolo Handshake para asegurar las comunicaciones.

Representación gráfica del funcionamiento de TLS, donde se ve el flujo de datos del protocolo Handshake, incluidas sus fases, y del protocolo de registro:

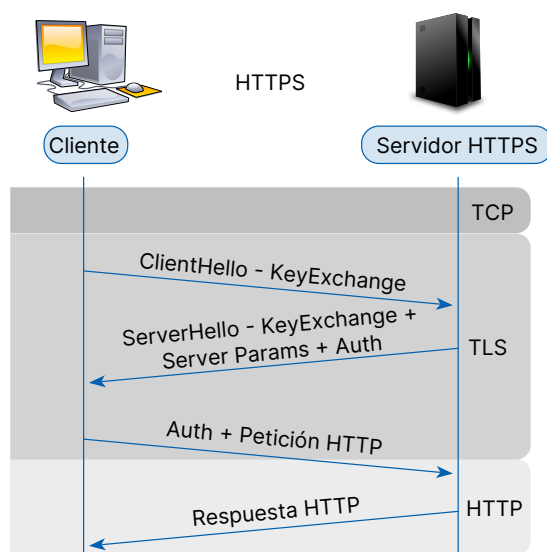


Figura 8. Funcionamiento del protocolo TLS.

04 Hosts virtuales

Mediante la utilización de los *hosts* virtuales se pueden tener alojados varios sitios web en un único servidor. Por defecto, cuando un servidor aloja únicamente un sitio web, lo más lógico es que este se configure en el directorio raíz del servidor y se acceda a través de la IP del servidor.

En el caso de querer alojar más de un sitio web en el mismo servidor, una opción es organizarlos por carpetas, de manera que, para acceder a cada uno de ellos, se escriba en la URL del navegador la IP del servidor, seguida de la carpeta correspondiente. Por ejemplo, si se tiene un servidor con tres sitios web y sus carpetas se llaman *tu_sitio1*, *tu_sitio2* y *tu_sitio3*, las URL para acceder a cada uno de ellos serán las siguientes:

`http://IP_servidor/tu_sitio1`

`http://IP_servidor/tu_sitio2`

`http://IP_servidor/tu_sitio3`

De esta forma, cuando el usuario escriba en su navegador `http://IP_servidor/tu_sitio1`, el servidor devolverá por defecto al navegador del cliente el archivo índice (.html, .php, etc.) contenido en la carpeta *tu_sitio1*. De igual manera se procederá para `http://IP_servidor/tu_sitio2` y `http://IP_servidor/tu_sitio3`.

Esto puede ser cómodo para un servidor propio de una empresa accesible a través de la red local, pero, en la práctica, los sitios web accesibles de forma pública, aunque estén en servidores compartidos, tienen asignado un nombre de dominio y se consultan a través de un puerto. Por tanto, debe existir un mecanismo que relacione el dominio, la IP y el puerto del servidor y la carpeta con los archivos. Para que el servidor sepa qué sitio web servir cuando le llega la petición del navegador del cliente, debe poder disponer de toda esta información para cada sitio web. Por eso, el servidor necesita guardar estos datos y lo hace en el archivo de *host* virtual correspondiente al sitio web en cuestión.

Existen varios tipos de *hosts* virtuales:

- » **Basados en IP:** el servidor tiene varias IP y cada sitio web tiene asignada una distinta.
- » **Basados en nombres:** el servidor tiene una única IP y todos los sitios web tienen asignada esa IP, pero un nombre de dominio diferente.
- » **Basados en puertos:** el servidor tiene una única IP y todos los sitios web tienen asignada la misma IP, pero un puerto distinto.

De las tres opciones, las que suelen ser más comunes son la basada en nombres y la basada en puertos. A continuación se detalla su funcionamiento.

4.1. Hosts virtuales basados en nombres

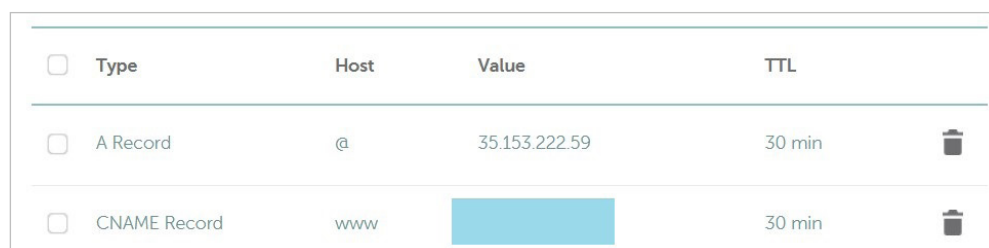
Ya se ha visto en apartados anteriores que, para relacionar cada dominio con la IP del servidor se utilizan los registros DNS. Pero, cuando le llega la petición al servidor, ¿cómo sabe este a qué carpeta ha de ir a buscar los archivos? Aquí es donde interviene el archivo de configuración del *host* virtual, que se almacena en el servidor y contiene la relación entre un dominio y la carpeta del sitio web asociada a él.

La versión 2 de Apache trabaja por defecto con *hosts* virtuales, ya que en el archivo `000-default.conf` se encuentra la configuración correspondiente para que cualquier petición web que llegue al servidor se redirija a la carpeta web por defecto. Este archivo se encuentra en la carpeta `/etc/apache2/sites-available`. Para crear más *hosts* virtuales, se puede hacer una copia de este archivo en esta misma carpeta y cambiarle la configuración necesaria. Una vez guardado, se tendrá que habilitar para que Apache lo utilice.

A continuación, se describen los pasos que hay que seguir para configurar el dominio *dominio1.com* para que acceda al sitio web alojado en el servidor en la carpeta *nombre_sitio1*:

PASO 1: Asociar dominio a IP del servidor

El primer paso es configurar el DNS a través de sus registros. En el caso de dominios comprados a través de un registrador oficial, este proporcionará acceso para modificar dichos registros y, si es el mismo proveedor de *hosting*, ya los habrá configurado de forma automática. A continuación se muestra un ejemplo de un posible panel de administración donde modificar esta información:



☐ Type	Host	Value	TTL	
☐ A Record	@	35.153.222.59	30 min	
☐ CNAME Record	www		30 min	

Figura 9. Panel de información DNS.

En este caso, al indicar el cliente un dominio en la dirección URL del servidor, la petición saldrá a Internet hasta llegar al servidor de DNS, éste devolverá la IP asociada al dominio y la petición seguirá a esa IP.

Además de cambiar la IP correspondiente al registro `www`, se pueden crear nuevos registros en otras IP, para, por ejemplo, hacer referencia a servidores web y de correo electrónico en máquinas separadas.

Hay que tener en cuenta que los servidores DNS asociados por defecto al dominio para resolver las peticiones serán los del propio registrador, aunque existe la posibilidad de cambiarlos. En este caso, los cambios en los registros se deberán indicar en el servidor de DNS correspondiente y no en el panel del propio *hosting*.

En el caso de querer configurar un dominio inventado o ficticio que realmente no esté registrado y que se utilice de forma local, al no poder resolverse su nombre por ningún servidor DNS, una opción es indicar la IP junto al nombre de dominio en el archivo `hosts` del cliente. Este archivo `hosts` es como un primer nivel de resolución. En caso de encontrar ahí el dominio asociado a una IP, la consulta no se realizará al servidor DNS configurado. La configuración debe indicarse en una línea nueva, como se ve en el siguiente ejemplo:

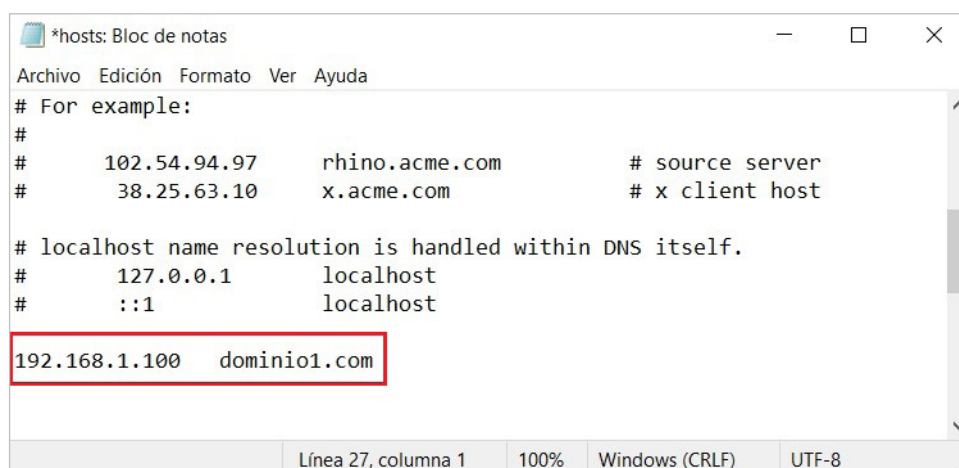


Figura 10. Resultado.

IMPORTANTE

Recuerda que el archivo **hosts** tiene diferentes ubicaciones que dependen del sistema operativo del servidor:

En sistemas operativos, Windows es `|Windows\System32\drivers\etc\hosts`

En sistemas operativos, GNU/Linux es: `/etc/hosts`

Según la configuración de Windows, si se abre el archivo **hosts** con el Bloc de notas, puede que, a la hora de guardar los cambios, aparezca un mensaje de que faltan permisos de administrador. Si se está en esta situación, hay que seguir los pasos descritos a continuación:

- » Abrir la consola de comandos a través de la opción *Ejecutar como administrador*, que aparece al hacer clic en el botón derecho del ratón.
- » Acceder a la carpeta `|Windows\System32\drivers\etc\hosts`.
- » Ejecutar el comando `notepad hosts`.

Cuando se tiene apuntado el dominio a la IP del servidor, bien sea a través de un proveedor de Internet o a través del archivo **hosts** del ordenador cliente, el siguiente paso es configurar en el servidor el vínculo que asocia el dominio con la carpeta. De esta forma, cuando llegue la petición al servidor, este sabrá en qué carpeta ha de buscar los archivos del sitio web. Este vínculo se establece a través de un archivo de configuración del servidor Apache.

PASO 2: Asociar dominio a carpeta web del servidor

A continuación, se describen los pasos que seguir para crear el archivo de configuración que vincula el dominio a la carpeta web en un servidor web con Apache instalado.

1. Acceder a la carpeta `sites-available`.

```
cd /etc/apache2/sites-available
```

2. Hacer una copia del archivo `000-default.conf` y llamarle `dominio1.com.conf`.

```
cp 000-default.conf dominio1.com.conf
```

3. Editar el archivo `dominio1.com.conf` para que quede así:

```
<VirtualHost *:80>
    ServerAdmin webmaster@dominio1.com
    ServerName dominio1.com
    ServerAlias dominio1.com
    DocumentRoot /var/www/html/tu_sitio1
    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined
</VirtualHost>
```

4. Habilitar el sitio.

```
a2ensite dominio1.com.conf
```

5. Reiniciar Apache.

```
systemctl restart apache2
```

NOTA: Si en algún momento se quiere deshabilitar este sitio, la instrucción que utilizar será:

```
a2dissite dominio1.com.conf
```

4.2. Hosts virtuales basados en puertos

Como se puede observar en el ejemplo anterior, la primera línea del *host* virtual indica el número de puerto a través del cual se accede al sitio web. Para configurar el puerto del sitio web basta con cambiarlo en esa línea del archivo de configuración e indicarlo en la URL. El puerto por defecto para el protocolo HTTP es el 80 y el puerto por defecto del protocolo HTTPS el 443. En estos casos no hará falta indicarlo en el navegador.

05 Hosting

Ya se ha visto en apartados anteriores que, para que un sitio web funcione, necesita que sus archivos estén almacenados en un ordenador que haga de servidor. Este servidor necesita tener instalados y disponibles de forma permanente algunos servicios; además, debe estar siempre a la escucha de posibles peticiones para servirlos en el menor tiempo posible.

También se ha visto con anterioridad que los servidores pueden alojar más de un sitio web y que, por tanto, estos deben estar identificados por un nombre de dominio.

Actualmente, hay muchas empresas que se dedican a alojar sitios web. La mayoría de ellas comparten los recursos de sus servidores entre todos sus clientes, incluso ofrecen planes en los que un único cliente puede tener alojados varios sitios web, cada uno con un dominio distinto que, como ya se ha mencionado antes, se asocia a la IP del servidor en el que está alojado el sitio web.

Hoy en día, cualquier persona o empresa puede registrar un dominio, es decir, pagar para poder utilizarlo y asociarlo a su sitio web. Este proceso de registro se debe hacer a través de empresas que están conectadas a una base de datos global para poder consultar si el dominio que se solicita está libre o, por el contrario, ya lo tiene registrado otro usuario. En este caso, no se podrá volver a registrar hasta que el usuario que lo utiliza deje de hacerlo. Esto es posible, ya que, cuando se registra un dominio, se hace por un tiempo limitado y, al finalizar este periodo, se puede renovar o liberar.

Al principio, solo unas pocas empresas denominadas **registradores de dominio** podían hacer el trámite para asociar un dominio a un propietario. Algunas extensiones de dominio se limitaban a residentes en unos países concretos y la empresa registradora, previa solicitud de la documentación correspondiente, decidía si conceder o no el nombre de dominio al solicitante.

Con el paso del tiempo, y a causa del gran crecimiento de Internet y de la demanda de estos servicios, el número de empresas que tienen capacidad para registrar dominios ha ido aumentando considerablemente y, en la actualidad, las empresas de *hosting* también ofrecen este servicio. También han aparecido en los últimos años nuevas extensiones de dominio y los requisitos para obtenerlos han cambiado.

Hoy es muy fácil obtener un dominio y un *hosting* con unos pocos clics. La ventaja en este caso es que, al terminar el proceso de contratación, el dominio elegido queda configurado y vinculado a la carpeta de los archivos del sitio web y es el servidor de DNS de la empresa de *hosting* quien se encarga de dirigir las peticiones que hagan los usuarios desde sus navegadores al nuevo dominio registrado. El propietario dispone de un panel de control, por lo general, fácil e intuitivo, para subir los archivos del sitio web. Así que, como se puede entender, cualquiera puede contratar un *hosting* y poner en marcha un sitio web accesible por todo el mundo en unos pocos clics y en cuestión de muy poco tiempo. Sobra decir que el contenido del sitio es responsabilidad de la persona o empresa que ha registrado el dominio y ha contratado el *hosting* y que, por tanto, debe utilizarlo de forma legal y hacer un buen uso.

Ahora mismo hay muchísimas empresas de *hosting*. Todas ofrecen varios planes para llegar a todas las necesidades de los clientes, desde planes prácticamente gratuitos hasta planes más caros con servicios avanzados. Incluso hay algunas que ofrecen planes gratuitos o de prueba para que los clientes puedan comprobar si el *hosting* se adapta a sus necesidades antes de contratarlo. Algunos ejemplos de estas empresas son:

» **000webhost:** el usuario se registra, crea su sitio y le asigna un nombre, que no tiene por qué ser un nombre de dominio; podría ser, por ejemplo, *mi_sitio_web*. Después, puede acceder al panel de control para subir los archivos de su sitio web a través del gestor de archivos y crear su base de datos a través del gestor de bases de datos. El sitio web será accesible a través de http://mi_sitio_web.000webhostapp.com.

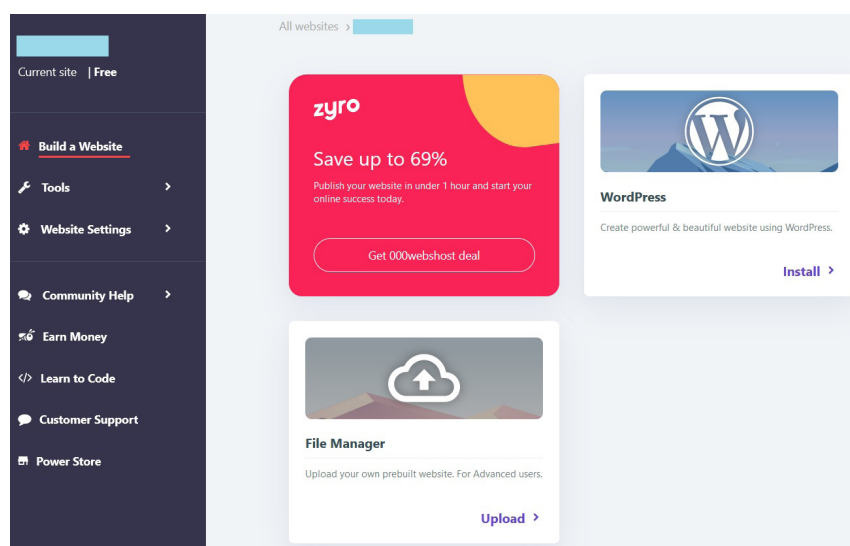
» **cdmon:** funciona de forma parecida, aunque sí solicita un nombre de dominio al registrar la plataforma de pruebas. Un ejemplo podría ser *midominio1.com*. También ofrece un panel para gestionar la base de datos y los archivos. El sitio web será accesible a través de la URL <http://midominio1.com.mialias.net/>, pero, en este caso, está protegido por un usuario y una contraseña que se proporciona desde el mismo panel de control y que el propietario del sitio deberá facilitar a quien quiera visitar su sitio web.

En ambos casos, el propietario puede pasarse a un plan de pago para eliminar las limitaciones de acceso en cualquier momento y tiene todas las opciones de configuración del *hosting* disponibles en el panel de control correspondiente, desde el cual, además de subir los archivos del sitio web y crear una base de datos, puede realizar otras acciones, como, por ejemplo, gestionar los registros DNS del dominio, crear cuentas de correo electrónico, elegir la versión de PHP sobre la que funcionará el sitio web, instalar aplicaciones como WordPress o Moodle de forma automática, etc. Las funciones disponibles en este panel de control dependen de la empresa de *hosting* y del plan que se haya contratado.

A continuación, se muestran algunas capturas de pantalla de los paneles de control de los dos *hostings* que se han nombrado.

Panel de control de 000webhost:

El panel de control de 000webhost tiene un menú en la parte izquierda de la pantalla a través del cual se puede acceder a las opciones disponibles. A continuación veremos algunas capturas de pantalla con algunas de las opciones:



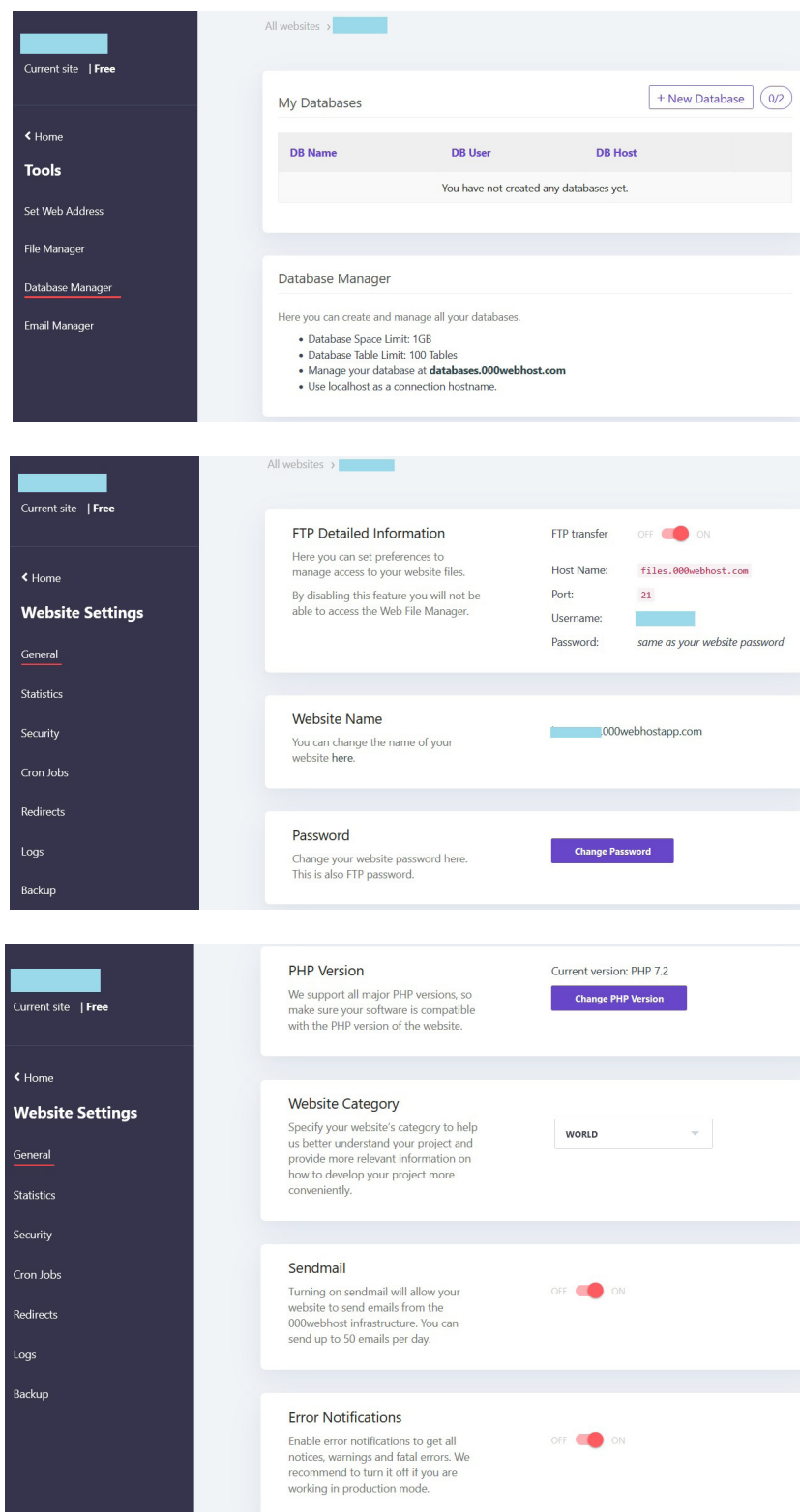


Figura 12. Panel de control de 000WEBHOST.

Panel de control de cdmon:

En cambio, el panel de control de cdmon tiene todas las opciones a la vista agrupadas por secciones debajo del área con los datos generales del *hosting*, como puede verse en las siguientes imágenes:

The screenshot displays the CDMON control panel interface. At the top, there's a navigation bar with 'Notificaciones', 'Renovaciones', and a search bar. Below this, a header section titled 'Gestionar hosting de la plataforma de pruebas' includes a breadcrumb trail and a 'Facturación' button. The main content area is divided into several sections:

- Visibilidad de su proyecto:** A section indicating that the project is not visible to the public and providing a 'Publicar' button.
- Datos de acceso:** A section showing login credentials (Usuario, Contraseña) and a link to 'Acceder a la página'.
- Información del hosting:** A table-like section with details such as 'Dominio asociado', 'Tipo de plan', 'Título', 'Versión PHP', 'Contraseña MySQL', 'IP web', 'Versión FTP', and 'Contraseña FTP'. Each row has a 'Cambiar' button.
- Correo electrónico:** A section with icons for 'Configuración del correo', 'Gestión de cuentas', 'Gestión de redirecciones', 'Acceso correo web (Roundcube)', 'Antivirus/Antispam', 'Usuario newsletter', and 'Listas Mailman'.
- Gestores de archivos:** A section with icons for 'FTP', 'Usuarios FTP', 'Acceso web FTP (net2ftp)', 'Clientes Git/SVN', 'SSH/SFTP', 'Descomprimir archivos', and 'Copia web comprimida'.
- Bases de datos:** A section with icons for 'MySQL', 'Usuarios MySQL', and 'Acceso bases de datos (phpMyAdmin)'.
- Servidor:** A section with icons for 'Configurar PHP', 'Información del servidor', 'Información del hosting', 'Configuración páginas de errores', 'Gon Jobs', and 'Logs: PHP y servidor web'.
- Domínios:** A section with icons for 'Redirecciones web', 'DNS', 'Cambiar dominio del hosting', 'Alias del hosting', 'Multidominios web', 'Subdominios', and 'Páginas de parking'.
- Estadísticas:** A section with icons for 'Estadísticas de visitas', 'Gráficas de consumo', and 'Acceso estadísticas (Awstat)'.
- Aplicaciones:** A section with icons for 'Instalar WordPress', 'Instalar Moodle', 'Instalar PrestaShop', 'Más aplicaciones', and 'Datos de acceso a las aplicaciones'.
- Seguridad:** A section with icons for 'Restaurar copia de seguridad', 'Permisos de archivos y carpetas', 'Firewall', and 'Certificados SSL'.
- Producto:** A section with icons for 'Publicar', 'Migración interna', 'Panel marca blanca (Gestiona)', 'Descuentos por volumen', and 'Cobros y renovaciones'.

Figura 13. Panel de control de CDMON.



VÍDEOLAB

Para consultar el siguiente vídeo sobre autenticación de aplicaciones, escanea el código QR o [pulsa aquí](#).

UAX FP